

Projeto Data Mining & Machine Learning

Aluno:

Daniel Barasini

Turma:

Data Mining & Machine Learning 2019 - BI_20 (Analytics)

Alvo de estudo:

Preço de Imóveis na região de Boston - EUA

Objetivo:

Estimar o preço do Imóvel baseado em suas características.

Descrição do Dataset:

Boston house prices dataset

****Data Set Characteristics:****

:Number of Instances: 506

:Number of Attributes: 13 numeric/categorical predictive. Median Value (attribute 14) is usually the target.

:Attribute Information (in order):

- CRIM per capita crime rate by town
- ZN proportion of residential land zoned for lots over 25,000 sq.ft.
- INDUS proportion of non-retail business acres per town
- CHAS Charles River dummy variable (= 1 if tract bounds river; 0 otherwise)
- NOX nitric oxides concentration (parts per 10 million)
- RM average number of rooms per dwelling
- AGE proportion of owner-occupied units built prior to 1940
- DIS weighted distances to five Boston employment centres
- RAD index of accessibility to radial highways
- TAX full-value property-tax rate per \$10,000
- PTRATIO pupil-teacher ratio by town
- B $1000(B_k - 0.63)^2$ where B_k is the proportion of blacks by town
- LSTAT % lower status of the population
- MEDV Median value of owner-occupied homes in \$1000's

:Missing Attribute Values: None

:Creator: Harrison, D. and Rubinfeld, D.L.

This is a copy of UCI ML housing dataset.

<https://archive.ics.uci.edu/ml/machine-learning-databases/housing/>

This dataset was taken from the StatLib library which is maintained at **Carnegie Mellon University**.

The Boston house-price data of Harrison, D. and Rubinfeld, D.L. 'Hedonic prices and the demand for clean air', J. Environ. Economics & Management, vol.5, 81-102, 1978. Used in Belsley, Kuh & Welsch, 'Regression diagnostics

...', Wiley, 1980. N.B. Various transformations are used in the table on pages 244-261 of the latter.

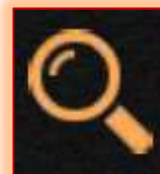
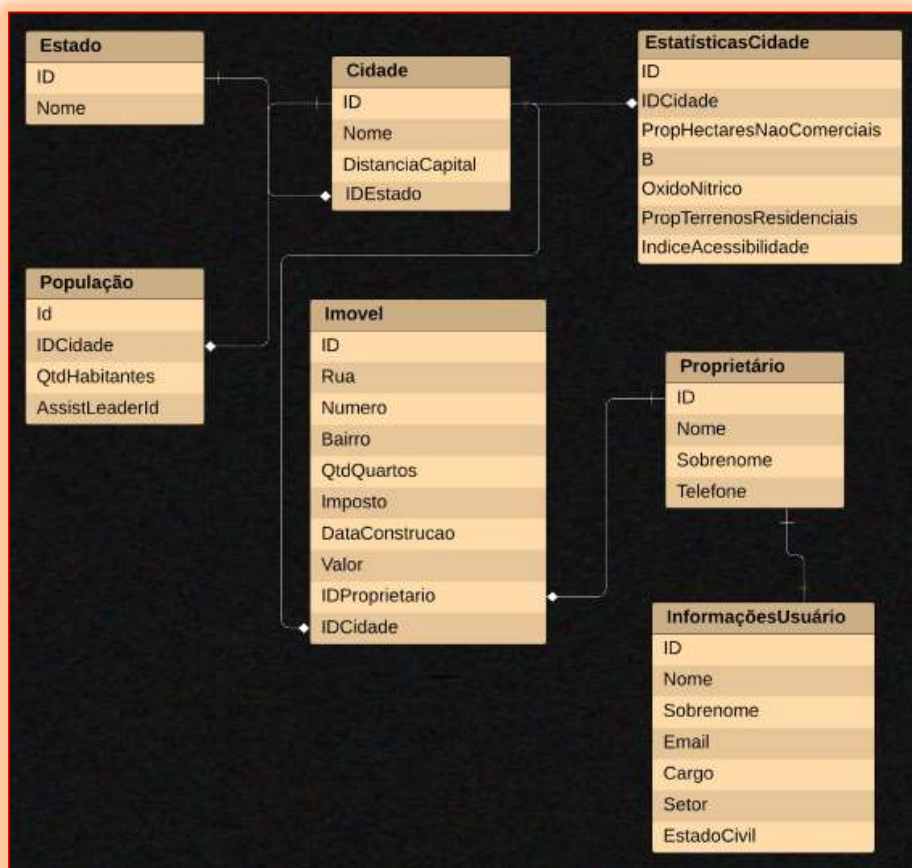
The Boston house-price data has been used in many machine learning papers that address regression problems.

.. topic:: References

- Belsley, Kuh & Welsch, 'Regression diagnostics: Identifying Influential Data and Sources of Collinearity', Wiley, 1980. 244-261.

- Quinlan,R. (1993). Combining Instance-Based and Model-Based Learning. In Proceedings on the Tenth International Conference of Machine Learning, 236-243, University of Massachusetts, Amherst. Morgan Kaufmann.

Diagrama Entidade e Relacionamento



Imoveis
PropHectaresNaoComerciais
PropPessoasDescAfro
OxidoNitrico
PropTerrenosResidenciais
IndiceAcessibilidade
Valor
TempoImovel
LimiteRio
QtdQuartos
Imposto

Conhecendo as variáveis da base

```
# imprime as 5 primeiras linhas do dataset  
data.head()
```

	CRIM	ZN	INDUS	CHAS	NOX	RM	AGE	DIS	RAD	TAX	PTRATIO	B	LSTAT
0	0.00632	18.0	2.31	0.0	0.538	6.575	65.2	4.0900	1.0	296.0	15.3	396.90	4.98
1	0.02731	0.0	7.07	0.0	0.469	6.421	78.9	4.9671	2.0	242.0	17.8	396.90	9.14
2	0.02729	0.0	7.07	0.0	0.469	7.185	61.1	4.9671	2.0	242.0	17.8	392.83	4.03
3	0.03237	0.0	2.18	0.0	0.458	6.998	45.8	6.0622	3.0	222.0	18.7	394.63	2.94
4	0.06905	0.0	2.18	0.0	0.458	7.147	54.2	6.0622	3.0	222.0	18.7	396.90	5.33

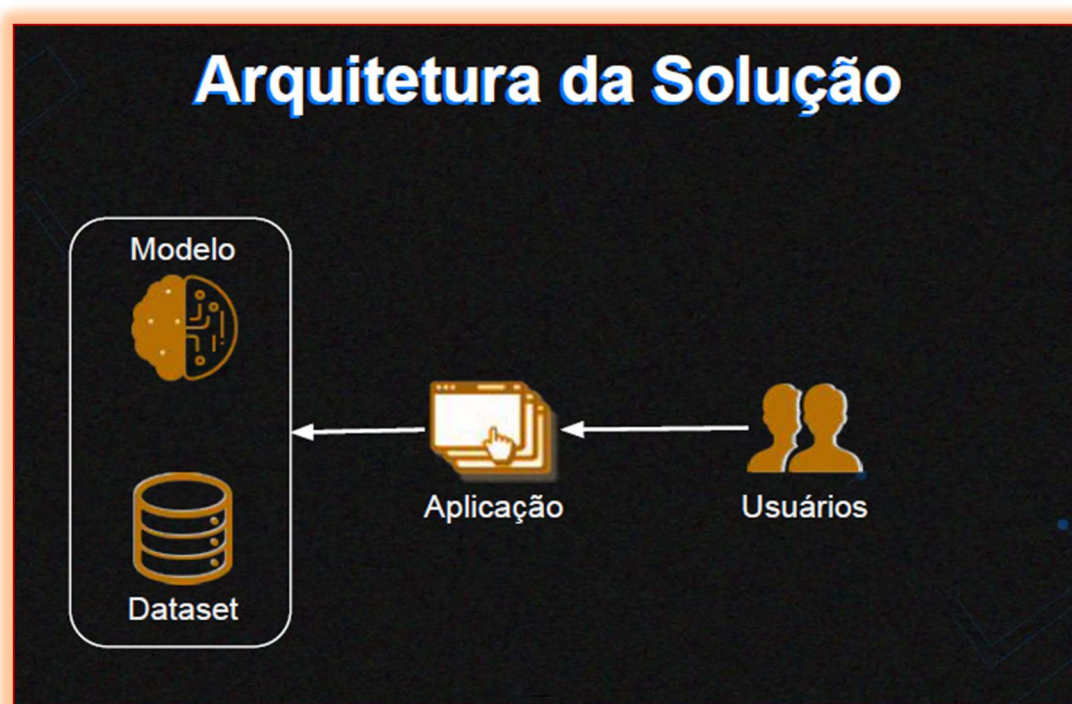
1. **CRIM:** Taxa de criminalidade per capita por região.
2. **ZN:** Proporção de terrenos residenciais divididos por lotes com mais de 25.000 pés quadrados.
3. **INDUS:** Essa é a proporção de hectares de negócios não comerciais por região.
4. **CHAS:** Variável fictícia Charles River (= 1 se o trecho limita o rio; 0 casos contrário)
5. **NOX:** Concentração de óxido nítrico (partes por 10 milhões)
6. **RM:** Número médio de quartos entre as casas do bairro
7. **Age:** Proporção de unidades ocupadas pelos proprietários construídas antes de 1940
8. **DIS:** Distância ponderada para cinco centros de emprego em Boston
9. **RAD:** Índice de acessibilidade às rodovias radiais
10. **IMPOSTO:** Taxa do imposto sobre a propriedade de valor total por US \$ 10.000
11. **B:** $1000 (B_k - 0,63)^2$, onde B_k é a proporção de pessoas de descendência afro-americana por região
12. **PTRATIO:** Bairros com maior proporção de alunos para professores (maior valor de 'PTRATIO')
13. **LSTAT:** Porcentagem de status mais baixo da população
14. **MEDV:** Valor médio de casas ocupadas pelos proprietários em US \$ 1000

Segue abaixo o Link para a pasta do Google Drive da turma de Data Mining & Machine Learning com o relatório de análise do Boston Data Set.

<https://drive.google.com/file/d/14XocmQxGC5QKlg8HI1VvoBe6Qss8eily/view?usp=sharing>

Observações

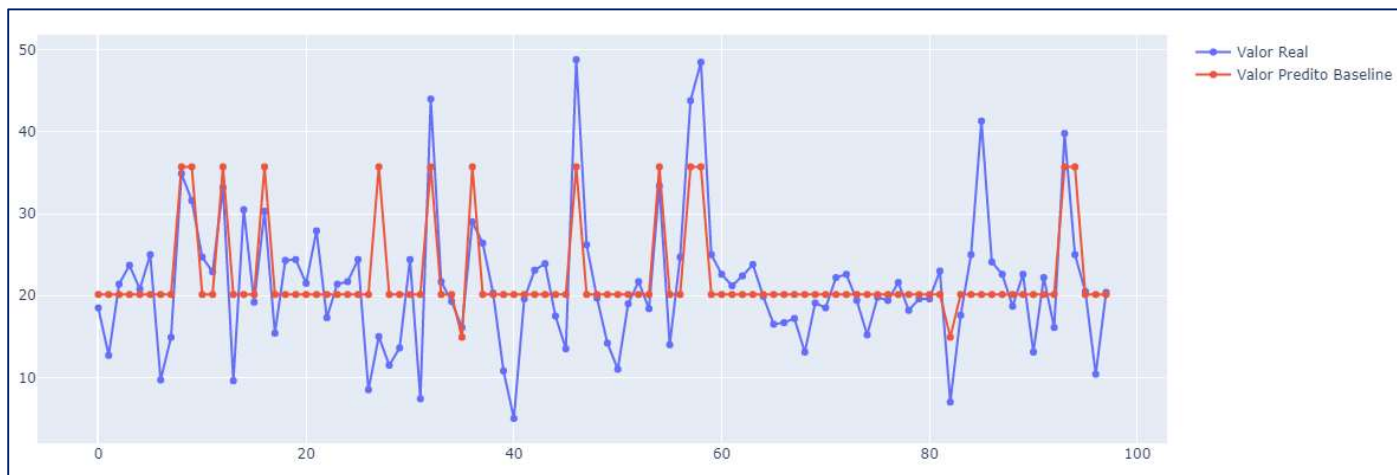
1. O coeficiente de correlação varia de -1 a 1. Se valor é próximo de 1, isto significa que existe uma forte correlação positiva entre as variáveis. Quando esse número é próximo de -1, as variáveis têm uma forte correlação negativa.
2. A relatório que executamos acima nos mostra que a nossa variável alvo (**MEDV**) é fortemente correlacionada com as variáveis **LSTAT** e **RM**.
3. **RAD** e **TAX** são fortemente correlacionadas, podemos remove-las do nosso modelo para evitar a multi-colinearidade.
4. O mesmo acontece com as variáveis **DIS** e **AGE** a qual tem a correlação de **-0.75**
5. A variável **ZN** possui **73%** de valores zero.



Resultados

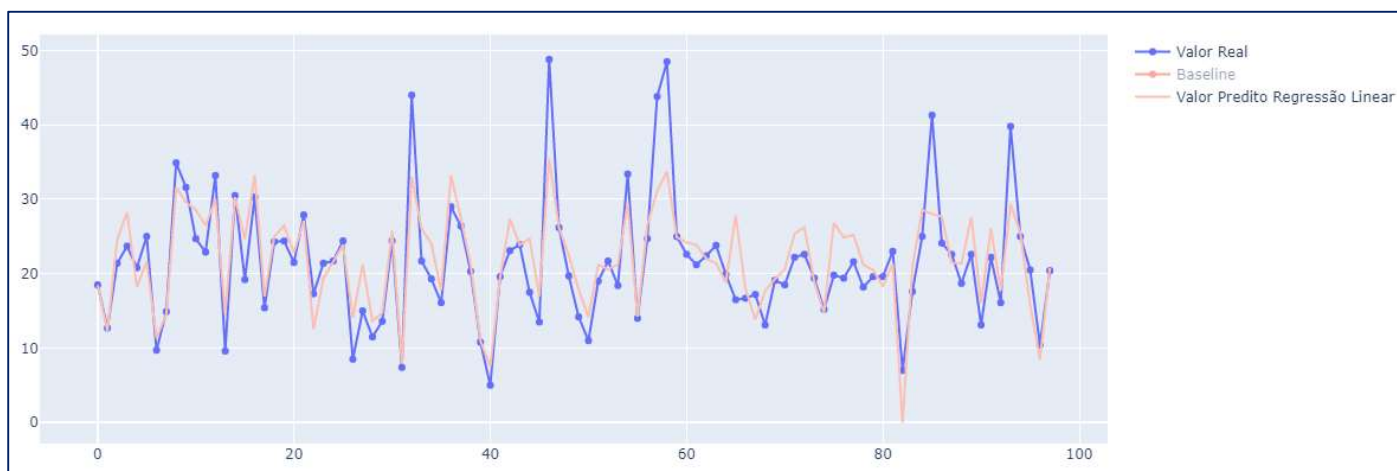
1. Performance do modelo **Baseline**

RMSE é: **6.205816494411828**



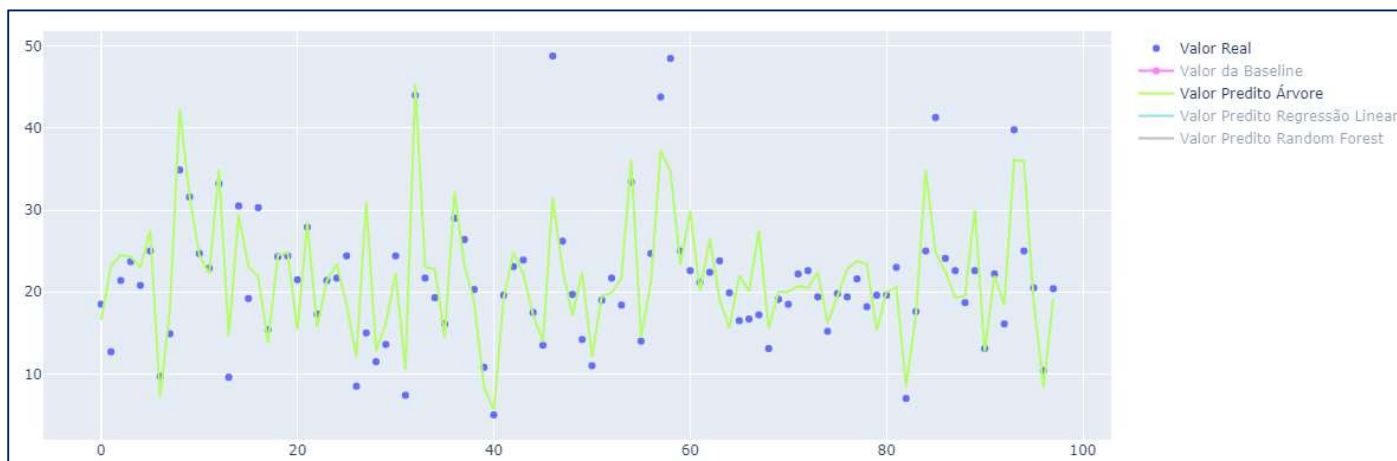
2. Performance do modelo **Regressão Linear**

RMSE é: **4.460277295153906**



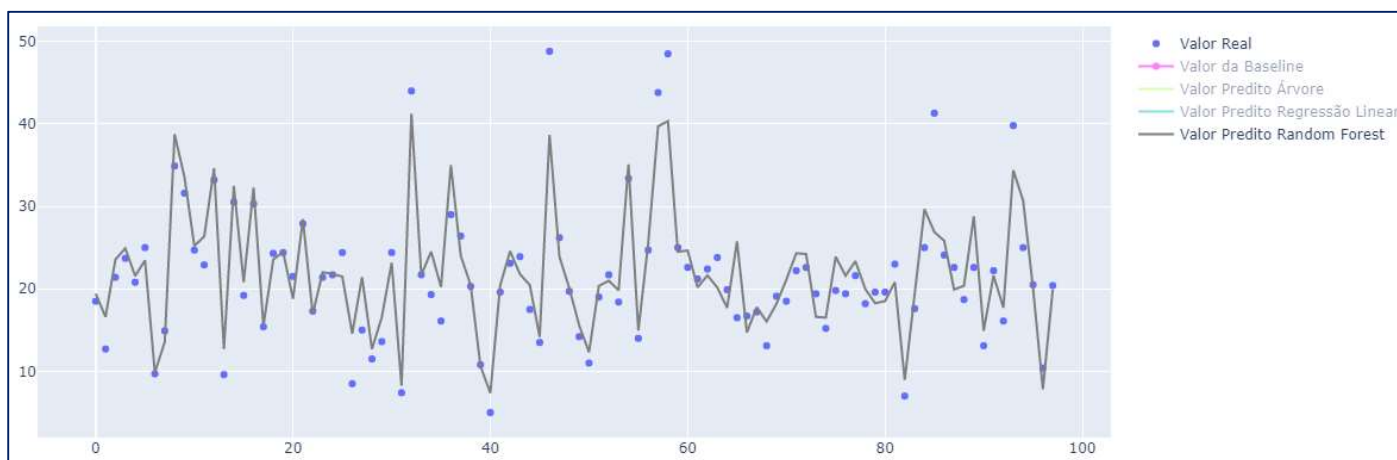
3. Performance do modelo **Arvore de Decisão**

RMSE é: **4.894280293550496**

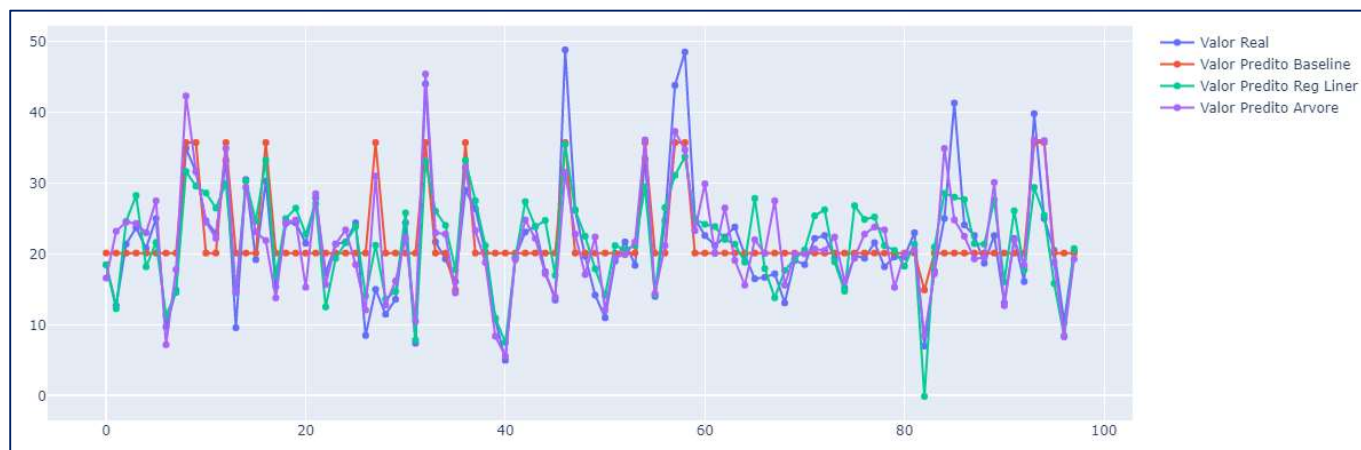


4. Performance do modelo **Random Forest**

RMSE é: **3.275683930767087**



Resultado Geral



Dataframe Resultado Geral.

```
# visualiza o dataframe df_results
df_results.head(10)
```

	valor_real	valor_predito_baseline	valor_predito_reg_linear	valor_predito_arvore
0	18.5	20.130425	18.459170	16.6
1	12.7	20.130425	12.279894	23.2
2	21.4	20.130425	24.588307	24.5
3	23.7	20.130425	28.254693	24.3
4	20.8	20.130425	18.195439	23.0
5	25.0	20.130425	21.630648	27.5
6	9.7	20.130425	11.319198	7.2
7	14.9	20.130425	14.561032	17.8
8	34.9	35.719231	31.623717	42.3
9	31.6	35.719231	29.588133	31.7

1. Performance modelo **Baseline** RMSE é: **6.205816494411828**
2. Performance modelo **Regressão Linear** RMSE é: **4.460277295153906**
3. Performance modelo **Arvore Decisão** RMSE é: **4.894280293550496**
4. Performance modelo **Random Forest** RMSE é: **3.275683930767087**

O melhor resultado de teste com o método Erro Médio Quadrático (RMSE) é o **Random Forest** com RMSE é: **3.275683930767087**, pois é o que mais se aproxima do valor real.

Sobre o Modelo - Random Forest

- ✓ Algoritmo supervisionado.
- ✓ Pode ser utilizado para classificação ou regressão.
- ✓ Dezenas de árvores combinadas para prever o melhor resultado.
- ✓ Aleatoriedade na seleção de atributos ao invés da seleção a partir do cálculo de impureza.
- ✓ Resolve o problema de overfitting da árvore de decisão.

Algumas vantagens

- ✓ Maior robustez
- ✓ Menos propenso a sofrer Overfitting em comparação com uma única Árvore de Decisão
- ✓ Permite a descoberta de conhecimento.
- ✓ Poucos parâmetros para ajustes.

Algumas desvantagens

- ✓ Exige um maior poder de processamento
- ✓ Pode ser lento o processo de classificação de novas amostras.

Como melhorar?

Tunning do Algoritmo de Machine Learning

Parâmetros da Random Forest

- O parâmetro `n_estimators` define a quantidade de árvores que serão usadas para criação dos modelos. Quanto maior o número de árvores, maior será o número de modelos no ensemble.
- O parâmetro `max_depth` define a profundidade da árvore, quanto mais profunda a árvore, mais especializado nos dados de treino o nosso modelo estará.
- O parâmetro `max_features` define a quantidade máxima de nós que serão usados para separar os níveis folha da árvore.

CODIGO

Conhecendo a base de dados

Monta o drive

```
from google.colab import drive
drive.mount('/content/drive')
```

Importando as bibliotecas básicas

```
import pandas as pd
import seaborn as sns
import numpy as np
import matplotlib.pyplot as plt
```

Carregando a Base de Dados

```
# carrega o dataset de london
from sklearn.datasets import load_boston
boston = load_boston()
```

```
# descrição do dataset
print (boston.DESCR)
```

```
# cria um dataframe pandas
data = pd.DataFrame(boston.data, columns=boston.feature_names)
```

```
# imprime as 5 primeiras linhas do dataset
data.head()
```

****Conhecendo as colunas da base de dados****

CRIM: Taxa de criminalidade per capita por regiao.

ZN: Proporção de terrenos residenciais divididos por lotes com mais de 25.000 pés quadrados.

INDUS: Essa é a proporção de hectares de negócios não comerciais por regiao.

CHAS: variável fictícia Charles River (= 1 se o trecho limita o rio; 0 caso contrário)

NOX: concentração de óxido nítrico (partes por 10 milhões)

RM: Número médio de quartos entre as casas do bairro

Age: proporção de unidades ocupadas pelos proprietários construídas antes de 1940

DIS: distâncias ponderadas para cinco centros de emprego em Boston

RAD: Índice de acessibilidade às rodovias radiais

IMPOSTO: taxa do imposto sobre a propriedade de valor total por US \$ 10.000

B: $1000 (B_k - 0,63)^2$, onde B_k é a proporção de pessoas de descendência afro-americana por regiao

PTRATIO: Bairros com maior proporção de alunos para professores (maior valor de 'PTRATIO')

LSTAT: porcentagem de status mais baixo da população

MEDV: valor médio de casas ocupadas pelos proprietários em US \$ 1000

****Adicionando a coluna que será nossa variável alvo****

```
# adiciona a variável MEDV
```

```
data['MEDV'] = boston.target
```

```
# imprime as 5 primeiras linhas do dataframe
```

```
data.head()
```

```
data.describe()
```

```
# filtra os top 16 maiores registro da coluna MEDV
```

```
top16 = data.nlargest(16, 'MEDV').index
```

```
# remove os valores listados em top16
```

```
data.drop(top16, inplace=True)
```

```
# Análise e Exploração dos Dados
```

****Nesta etapa nosso objetivo é conhecer os dados que estamos trabalhando.****

Usaremos Pandas Profiling para essa etapa:

```
# instalando o pandas profiling
```

```
!pip install https://github.com/pandas-profiling/pandas-profiling/archive/master.zip
```

```
# import o ProfileReport
```

```
from pandas_profiling import ProfileReport
```

```
# executando o profile
```

```
profile = ProfileReport(data, title='Relatório - Pandas Profiling', html={'style':{'full_width':True}})
```

```
profile
```

****Observações****

1. O coeficiente de correlação varia de -1 a 1. Se valor é próximo de 1, isto significa que existe uma forte correlação positiva entre as variáveis. Quando esse número é próximo de -1, as variáveis tem uma forte correlação negativa.

2. O relatório que executamos acima nos mostra que a nossa variável alvo (MEDV) é fortemente correlacionada com as variáveis LSTAT e RM

3. RAD e TAX são fortemente correlacionadas, podemos remove-las do nosso modelo para evitar a multicolinearidade.
4. O mesmo acontece com colunas DIS and AGE a qual tem a correlação de -0.75
5. A coluna ZN possui 73% de valores zero.

```
# salvando o relatório no disco
profile.to_file(output_file="Relatorio01.html")

# **Definindo um Baseline**

- 'Uma baseline é importante para ter marcos no projeto'.
- 'Permite uma explicação fácil para todos os envolvidos'.
- 'É algo que sempre tentaremos ganhar na medida do possível'.

# converte os dados
data.RM = data.RM.astype(int)

data.info()

# estatística descritiva da coluna numero de quartos
data.RM.describe()

# definindo a regra para categorizar os dados
categorias = []

# alimenta a lista categorias
for i in data.RM.iteritems():
    valor = (i[1])
    if valor <= 4:
        categorias.append('Pequeno')
    elif valor < 7:
        categorias.append('Medio')
    else:
        categorias.append('Grande')

# cria a coluna categorias
data['categorias'] = categorias

# imprime a contagem de categorias
data.categorias.value_counts()

# agrupa as categorias e calcula as médias
medias_categorias = data.groupby(by='categorias')['MEDV'].mean()

# visualizando a variável medias_categorias
medias_categorias

# criando o dicionario com chaves medio, grande e pequeno e seus valores
```

```

dic_baseline = {'Grande': medias_categorias[0], 'Medio': medias_categorias[1], 'Pequeno':
medias_categorias[2]}

# imprime dicionario
dic_baseline

# cria a função retorna baseline
def retorna_baseline(num_quartos):
    if num_quartos <= 4:
        return dic_baseline.get('Pequeno')
    elif num_quartos < 7:
        return dic_baseline.get('Medio')
    else:
        return dic_baseline.get('Grande')

# chama a função retorna baseline
retorna_baseline(7)

# imprime as 5 primeiras linhas do dataframe
data.head()

for i in data.RM.iteritems():
    n_quartos = i[1]
    print('Número de quartos é: {} , Valor médio: {}'.format(n_quartos,retorna_baseline(n_quartos)))

# **Trabalhando com a baseline**

### *Preparando os dados*

# remove as colunas colineares, variavel alvo e a coluna categorias
X = data.drop(['RAD','TAX','MEDV','DIS','AGE','ZN','categorias'], axis=1)

# variável 'y' é nossa variável alvo
y = data['MEDV']

# Dados na variável X
X.head()

# Variável alvo
y.head()

## *Separando os conjuntos de treino e teste*

# carrega o método train_test_split
from sklearn.model_selection import train_test_split

# divide os dados entre o conjunto de treino e teste, 80% e 20% respectivamente.
# define qualquer valor para o parâmetro random_state.
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.2, random_state=5)

# imprime a quantidade de linhas dos conjuntos
print ('X_train: numero de linhas e colunas: {}'.format(X_train.shape))
print ('X_test: numero de linhas e colunas: {}'.format(X_test.shape))
print ('y_train: numero de linhas e colunas: {}'.format(y_train.shape))
print ('y_test: numero de linhas e colunas: {}'.format(y_test.shape))

```

```

## *Modelo de baseline*

# imprimindo as 5 primeiras linhas dos dados de teste
X_test.head()

# definindo uma lista de predicoes
predicoes = []

# para cada elemento do teste alimenta a lista predicoes
for i in X_test.RM.iteritems():
    n_quartos = i[1]
    predicoes.append(retorna_baseline(n_quartos))

# imprime as 10 primeiras linhas da lista de predicoes
predicoes[:10]

***Atribuindo resultados a um dataframe***

- `Manteremos um log ao longo do projeto`
- `Permite a evolução dado o baseline`

# cria um dataframe vazio
df_results = pd.DataFrame()

# adiciona a coluna valor_real
df_results['valor_real'] = y_test.values

# cria a coluna valor_predito_baseline com as predicoes
df_results['valor_predito_baseline'] = predicoes

# imprimindo as 10 primeiras linhas do df_results
df_results.head(10)

#### *Plota o valor real e o baseline*

import plotly.graph_objects as go

# Create traces
fig = go.Figure()

# Linha com os dados de teste
fig.add_trace(go.Scatter(x=df_results.index,
                        y=df_results.valor_real,
                        mode='lines+markers',
                        name='Valor Real'))

# Linha com os dados preditos
fig.add_trace(go.Scatter(x=df_results.index,
                        y=df_results.valor_predito_baseline,
                        mode='lines+markers',
                        name='Valor Predito Baseline'))

# Plota a figura

```

```
fig.show()
```

```
### *Avaliação do modelo baseline*
```

```
# calcula a métrica rmse
```

```
from sklearn.metrics import mean_squared_error
```

```
from math import sqrt
```

```
rmse = (np.sqrt(mean_squared_error(y_test, predicoes)))
```

```
# imprime a performance do modelo
```

```
print ('Performance do modelo baseline:')
```

```
print("\nRMSE é: {}".format(rmse))
```

```
### Machine Learning
```

```
#### Regressão Linear
```

```
- `Devido a correlação entre algumas features podemos usar uma regressão linear.`
```

```
- `Modelo simples e eficiente.`
```

```
- `Solução rápida e robusta.`
```

```
# carrega o pacote LinearRegression
```

```
from sklearn.linear_model import LinearRegression
```

```
# cria o objeto do tipo LinearRegression
```

```
lin_model = LinearRegression()
```

```
# treina o algoritmo de regressão linear
```

```
lin_model.fit(X_train, y_train)
```

```
# avaliação do modelo nos dados de teste
```

```
y_pred = lin_model.predict(X_test)
```

```
# calcula a métrica rmse
```

```
rmse = (np.sqrt(mean_squared_error(y_test, y_pred)))
```

```
# imprime a performance do modelo
```

```
print ('Performance do modelo avaliado com os dados de teste:')
```

```
print("\nRMSE é: {}".format(rmse))
```

```
# atribui os resultados no dataframe df_results
```

```
df_results['valor_predito_reg_linear'] = lin_model.predict(X_test)
```

```
df_results.head(10)
```

```
import plotly.graph_objects as go
```

```
# Create traces
```

```
fig = go.Figure()
```

```
# Linha com os dados de teste
```

```
fig.add_trace(go.Scatter(x=df_results.index,  
                        y=df_results.valor_real,  
                        mode='lines+markers',
```

```

        name='Valor Real'))

# Linha com os dados de baseline
fig.add_trace(go.Scatter(x=df_results.index,
                        y=df_results.valor_predito_baseline,
                        mode='lines+markers',
                        name='Baseline'))

# Linha com os dados preditos pela regressão linear
fig.add_trace(go.Scatter(x=df_results.index,
                        y=df_results.valor_predito_reg_linear,
                        mode='lines',
                        line = dict(color = '#FEBFB3'),
                        name='Valor Predito Regressão Linear'))

# Plota a figura
fig.show()

##### Arvore de decisão

* `Algoritmo de aprendizado supervisionado`
* `Explicabilidade do Modelo`
* `Simples`

# importa o pacote DecisionTreeRegressor
from sklearn.tree import DecisionTreeRegressor

# cria um objeto do tipo DecisionTreeRegressor
regressor = DecisionTreeRegressor()

# treina o algoritmo
regressor.fit(X_train, y_train)

Avaliando o modelo de Árvore de Decisão

# fazendo as previsões
y_pred = regressor.predict(X_test)

# Adicionando o valor do modelo no dataframe df_results
df_results['valor_predito_arvore'] = y_pred

# visualiza o dataframe df_results
df_results.head(10)

import plotly.graph_objects as go

# cria uma figura
fig = go.Figure()

# Linha com os dados de teste
fig.add_trace(go.Scatter(x=df_results.index,
                        y=df_results.valor_real,
                        mode='lines+markers',
                        name='Valor Real'))

```

```

# Linha com os dados de teste
fig.add_trace(go.Scatter(x=df_results.index,
                        y=df_results.valor_predito_baseline,
                        mode='lines+markers',
                        name='Valor Predito Baseline'))

# Linha com os dados de teste
fig.add_trace(go.Scatter(x=df_results.index,
                        y=df_results.valor_predito_reg_linear,
                        mode='lines+markers',
                        name='Valor Predito Reg Liner'))

# Linha com os dados preditos
fig.add_trace(go.Scatter(x=df_results.index,
                        y=df_results.valor_predito_arvore,
                        mode='lines+markers',
                        name='Valor Predito Arvore'))

# Plota a figura
fig.show()

```

Avaliação do Modelo

```

# calcula a métrica rmse
rmse = (np.sqrt(mean_squared_error(y_test, y_pred)))

# imprime a performance do modelo
print ('Performance do modelo avaliado com os dados de teste:')
print("\nRMSE é: {} '.format(rmse))

```

Random Forest

- 'Método Ensemble'
- 'Mais robustez para os modelos'

```

# Importa o método RandomForestRegressor
from sklearn.ensemble import RandomForestRegressor

# cria o objeto rf_regressor
rf_regressor = RandomForestRegressor()

# treina o algoritmo
rf_regressor.fit(X_train, y_train)

# avaliação do modelo nos dados de teste
y_pred = rf_regressor.predict(X_test)

# adiciona os resultados no dataframe df_results
df_results['valor_predito_random_forest'] = rf_regressor.predict(X_test)

# calcula a métrica rmse
rmse = (np.sqrt(mean_squared_error(y_test, y_pred)))

# imprime a performance do modelo

```

```
print ('Performance do modelo avaliado com os dados de teste:')
print('\nRMSE é: {}'.format(rmse))
```

```
# plota os resultados dos modelos e o valor real.
import plotly.graph_objects as go
```

```
# Create traces
fig = go.Figure()
```

```
# Linha com os dados de teste
fig.add_trace(go.Scatter(x=df_results.index,
                        y=df_results.valor_real,
                        mode='markers',
                        name='Valor Real'))
```

```
# Linha com os dados da baseline
fig.add_trace(go.Scatter(x=df_results.index,
                        y=df_results.valor_predito_baseline,
                        mode='lines+markers',
                        line = dict(color = '#FF00FF'),
                        name='Valor da Baseline'))
```

```
# Linha com os dados preditos pela arvore
fig.add_trace(go.Scatter(x=df_results.index,
                        y=df_results.valor_predito_arvore,
                        mode='lines',
                        line = dict(color = '#B2FF66'),
                        name='Valor Predito Árvore'))
```

```
# Linha com os dados preditos pela regressão linear
fig.add_trace(go.Scatter(x=df_results.index,
                        y=df_results.valor_predito_reg_linear,
                        mode='lines',
                        line = dict(color = '#17BECF'),
                        name='Valor Predito Regressão Linear'))
```

```
# Linha com os dados preditos pela Random Forest
fig.add_trace(go.Scatter(x=df_results.index,
                        y=df_results.valor_predito_random_forest,
                        mode='lines',
                        line = dict(color = '#7F7F7F'),
                        name='Valor Predito Random Forest'))
```

```
# Plota a figura
fig.show()
```

```
#### Tuning do Algoritmo de Machine Learning
```

```
**Parâmetros da Random Forest**
```

- 'O parâmetro `n_estimators` define a quantidade de árvores que serão usadas para criação dos modelos. Quanto maior o número de árvores, maior será o número de modelos no ensemble.'

- 'O parâmetro `max_depth` define a profundidade da árvore, quanto mais profunda a árvore, mais especializado nos dados de treino o nosso modelo estará.'

- 'O parâmetro `max_features` define a quantidade máxima de nós que serão usados para separar os níveis folha da árvore'

Deploy da Aplicação

```
X['MEDV'] = y
```

```
X.head()
```

```
X.to_csv('data.csv', index=False)
```

Relatório - Pandas Profiling